

✓ Lambda Functions in Python

A Python Buddy Guide

Lambda functions are small, anonymous functions defined with the `lambda` keyword.

They are useful when you need a short function for a single use — especially as arguments to `map()`, `filter()`, `sorted()`, and `reduce()`.

This notebook covers:

1. What is a lambda function?
 2. Syntax and basic examples
 3. Lambda vs `def`
 4. Using lambda with `map()`
 5. Using lambda with `filter()`
 6. Using lambda with `sorted()`
 7. Using lambda with `reduce()`
 8. Lambda in `max()` and `min()`
 9. Applied example — OPPE question walkthrough
-

✓ 1. What is a Lambda Function?

A **lambda function** is a function with no name. It is defined inline and can take any number of arguments, but has exactly **one expression** as its body.

```
lambda arguments : expression
```

The expression is evaluated and returned automatically — no `return` keyword needed.

```
# A regular function
def square(x):
    return x ** 2

# The same function as a lambda
square_lambda = lambda x: x ** 2

print(square(5))          # 25
print(square_lambda(5))   # 25
```

✓ 2. Syntax and Basic Examples

```
# Zero arguments
greet = lambda: "Hello, World!"
print(greet())
```

```
# One argument
double = lambda x: x * 2
print(double(7)) # 14
```

```
# Two arguments
add = lambda x, y: x + y
print(add(3, 4)) # 7
```

```
# With a condition (ternary)
is_even = lambda n: "even" if n % 2 == 0 else "odd"
print(is_even(4)) # even
print(is_even(7)) # odd
```

```
# Calling a lambda immediately (without assigning it)
result = (lambda x, y: x ** 2 + y ** 2)(3, 4)
print(result) # 25
```

✓ 3. Lambda vs `def`

	<code>def</code>	<code>lambda</code>
Name	Has a name	Anonymous
Body	Multiple statements	Single expression only
<code>return</code>	Explicit	Implicit
Reusability	Easy to reuse	Typically single-use
Readability	Better for complex logic	Better for short inline use

Rule of thumb: if a lambda is getting complex or needs more than one line of logic, use `def` instead.

```
# When NOT to use lambda – this is hard to read
f = lambda x: x**3 - 2*x**2 + 4*x - 8 # use def for complex expressions

# Better as a def
def polynomial(x):
    return x**3 - 2*x**2 + 4*x - 8

print(f(2), polynomial(2)) # both give 0
```

✓ 4. Lambda with `map()`

`map(function, iterable)` applies a function to **every element** of an iterable.

```
[a, b, c] → map(f, ...) → [f(a), f(b), f(c)]
```

```
nums = [1, 2, 3, 4, 5]

# Square every number
squared = list(map(lambda x: x ** 2, nums))
print(squared) # [1, 4, 9, 16, 25]
```

```
# Convert strings to uppercase
words = ["apple", "banana", "cherry"]
upper = list(map(lambda w: w.upper(), words))
print(upper) # ['APPLE', 'BANANA', 'CHERRY']
```

```
# map() with two iterables
a = [1, 2, 3]
b = [10, 20, 30]
sums = list(map(lambda x, y: x + y, a, b))
print(sums) # [11, 22, 33]
```

✓ 5. Lambda with `filter()`

`filter(function, iterable)` keeps only the elements where the function returns `True`.

```
[a, b, c, d] → filter(cond, ...) → [elements where cond is True]
```

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]

# Keep only even numbers
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # [2, 4, 6, 8]
```

```
# Keep words longer than 4 characters
words = ["hi", "hello", "hey", "howdy", "yo"]
long_words = list(filter(lambda w: len(w) > 4, words))
print(long_words) # ['hello', 'howdy']
```

```
# Combining filter + map: square only the even numbers
nums = [1, 2, 3, 4, 5, 6]
result = list(map(lambda x: x**2, filter(lambda x: x % 2 == 0, nums)))
print(result) # [4, 16, 36]
```

✓ 6. Lambda with `sorted()`

`sorted(iterable, key=function)` sorts by a custom key.

The `key` function is applied to each element before comparison — the result is used for sorting, not the element itself.

```
words = ["banana", "apple", "fig", "cherry"]

# Sort by length
print(sorted(words, key=lambda w: len(w)))
# ['fig', 'apple', 'banana', 'cherry']
```

```
# Sort list of tuples by second element
students = [("Alice", 88), ("Bob", 72), ("Carol", 95)]
by_marks = sorted(students, key=lambda s: s[1], reverse=True)
print(by_marks)
# [('Carol', 95), ('Alice', 88), ('Bob', 72)]
```

```
# Sort by multiple criteria: first by length, then alphabetically
words = ["cat", "apple", "ant", "avocado", "bat"]
print(sorted(words, key=lambda w: (len(w), w)))
# ['ant', 'bat', 'cat', 'apple', 'avocado']
```

✓ 7. Lambda with `reduce()`

`reduce(function, iterable)` from `functools` applies a function **cumulatively** — combining all elements into a single value.

```
[a, b, c, d] → reduce(f, ...) → f(f(f(a, b), c), d)
```

```
from functools import reduce

nums = [1, 2, 3, 4, 5]

# Sum all elements
total = reduce(lambda acc, x: acc + x, nums)
print(total) # 15
```

```
# Product of all elements
product = reduce(lambda acc, x: acc * x, nums)
print(product) # 120
```

```
# Find the maximum using reduce
maximum = reduce(lambda a, b: a if a > b else b, nums)
print(maximum) # 5
```

```
# Horner's method for polynomial evaluation – a classic reduce use
# Evaluate  $2x^3 + 3x^2 + 5x + 7$  at  $x = 2$ 
coef = [2, 3, 5, 7]
x = 2
result = reduce(lambda acc, c: acc * x + c, coef)
print(result) # 45
```

✓ 8. Lambda in `max()` and `min()`

`max()` and `min()` also accept a `key` argument — works exactly like `sorted()`.

```
words = ["banana", "apple", "fig", "cherry"]

# Longest word
print(max(words, key=lambda w: len(w))) # banana

# Shortest word
print(min(words, key=lambda w: len(w))) # fig
```

```
# Tie-breaking with tuple key
# Highest score; ties broken by name alphabetically
students = [("Alice", 88), ("Bob", 95), ("Carol", 95)]
top = min(students, key=lambda s: (-s[1], s[0]))
print(top) # ('Bob', 95) – Bob comes before Carol alphabetically
```

✓ 9. Applied Example — OPPE Question Walkthrough

2025 May Set 3 — S3Q1: Batsman Performance Analysis

This problem asks for five functions on cricket career data.

It's a great showcase for lambda because every function has a natural functional form.

Data format:

```
batsman_data = {
    year (int): [list of runs scored in each match]
```

```
}
```

```
# Sample data
data = {
    2016: [88, 66, 130, 122, 117, 95, 86],
    2017: [149, 66, 110],
    2018: [157, 84],
    2019: [148, 127, 71, 117],
    2020: [91, 156, 80, 135, 152, 109]
}
```

```
# — Function 1: overall_run_stats —————
# Flatten all runs into one list, compute min/max/total/average

def overall_run_stats(batsman_data: dict) -> dict:
    all_runs = [r for runs in batsman_data.values() for r in runs]
    return {
        "min":    min(all_runs),
        "max":    max(all_runs),
        "total":  sum(all_runs),
        "average": round(sum(all_runs) / len(all_runs))
    }

print(overall_run_stats(data))
```

```
# — Function 2: century_rate —————
# century_rate = (centuries / matches) * 100
# Lambda: filter centuries, count, divide

def century_rate(runs: list) -> int:
    return round(len(list(filter(lambda r: r >= 100, runs))) / len(runs) * 100)

# Breaking it down:
# filter(lambda r: r >= 100, runs) → keeps only scores >= 100
# len(...)                        → counts centuries
# / len(runs) * 100               → as a percentage
# round(...)                      → to nearest int

print(century_rate(data[2016])) # 43
print(century_rate(data[2017])) # 67
```

```
# — Function 3: average_yearly_century_rate —————
# Compute century_rate for each year, then average
# Lambda: map century_rate over all years' runs

def average_yearly_century_rate(batsman_data: dict) -> int:
    rates = list(map(lambda runs: century_rate(runs), batsman_data.values()))
    return round(sum(rates) / len(rates))

# map(lambda runs: century_rate(runs), ...) applies century_rate to each ye
```

```
avg_rate = average_yearly_century_rate(data)
print(avg_rate) # 60
```

```
# — Function 4: years_with_more_than_average_yearly_century_rate
# Return years where century_rate > average
# Lambda: filter years by their century rate

def years_with_more_than_average_yearly_century_rate(batsman_data: dict) ->
    avg = average_yearly_century_rate(batsman_data)
    return set(filter(lambda y: century_rate(batsman_data[y]) > avg, batsman_data.keys()))

# filter iterates over keys (years)
# lambda y: century_rate(batsman_data[y]) > avg checks each year's rate

print(years_with_more_than_average_yearly_century_rate(data))
# {2017, 2019, 2020}
```

```
# — Function 5: year_with_most_average_runs —————
# Return year with highest average runs; ties -> earliest year
# Lambda: key for max - negate average, then year for tie-break

def year_with_most_average_runs(batsman_data: dict) -> int:
    return min(
        batsman_data,
        key=lambda y: (-sum(batsman_data[y]) / len(batsman_data[y]), y)
    )

# lambda y: (-avg_runs, y)
# Negating avg_runs makes min() act like max() for averages
# Appending y as second tuple element breaks ties by earliest year

print(year_with_most_average_runs(data)) # 2018
```

Summary of lambda patterns used in this problem

Function	Lambda pattern	What it does
<code>century_rate</code>	<code>filter(lambda r: r >= 100, runs)</code>	Keeps only centuries
<code>average_yearly_century_rate</code>	<code>map(lambda runs: century_rate(runs), ...)</code>	Applies rate function to each century
<code>years_with_more_than_average</code>	<code>filter(lambda y: rate(y) > avg, data)</code>	Filters years by rate
<code>year_with_most_average_runs</code>	<code>min(..., key=lambda y: (-avg(y), y))</code>	Finds max with tie-break

Key Takeaways

Concept	When to use
<code>map(lambda ...)</code>	Transform every element
<code>filter(lambda ...)</code>	Keep elements matching a condition
<code>sorted(key=lambda ...)</code>	Sort by a custom rule
<code>reduce(lambda ...)</code>	Accumulate elements into one value
<code>max/min(key=lambda ...)</code>	Find best element by a custom rule

Concept	When to use
Tuple key <code>lambda x: (a, b)</code>	Sort/compare by multiple criteria
Negate for reverse <code>lambda x: -f(x)</code>	Use <code>min()</code> to find the maximum

Remember: lambda is syntax sugar for a simple `def`. If your lambda is getting hard to read — rewrite it as `def`.